

CPS 499/564 Project 1 Technical Report

Team Members:

Benjamin Merrell, Elijah Rose, and Ashley Blankenship.

Solutions

Problem 1:

Problem 1a) Detecting Tampering

To determine whether a color channel has been manipulated, I combined two strategies: Histogram Analysis and noise estimation. Each strategy gives each channel a score, and I normalize the scores so they sum to 1 across all three channels. In total, we use three tests; each test contributes $\frac{1}{3}$ to the total confidence score. Two histogram tests contribute about 67% of the score, and the noise test contributes the remaining 33%.

Histogram Analysis:

The first test is used to estimate a score by comparing the overall shapes of the histograms. Since most of the normal channels tended to have similar histogram shapes. I first normalized the histograms so they sum up to 1. I then took the difference between each pair of channels, summed the absolute differences, and divided them by 2 (this normalizes it). Each channel's score is the average of its differences between the other channels, so a manipulated channel would have a higher average difference than the unmodified channels. Finally, the scores are normalized by dividing each score by the total of all of the scores so they sum to 1.

Code:

```
% Gets normalized histogram of all channels (sum of histogram = 1)
hR = imhist(R) / numel(R);
hG = imhist(G) / numel(G);
hB = imhist(B) / numel(B);
% Sums the difference in histograms and normalizes it
dRG = sum(abs(hR - hG)) / 2;
dRB = sum(abs(hR - hB)) / 2;
dGB = sum(abs(hG - hB)) / 2;
% Calculates the average difference per channel
R_hist_analysis_diff = (dRG + dRB) / 2;
G_hist_analysis_diff = (dRG + dGB) / 2;
B_hist_analysis_diff = (dRB + dGB) / 2;
% Takes those differences and puts them into a ratio. The total of all
%ratio channels equals 1.
hist_analysis_total = R_hist_analysis_diff + G_hist_analysis_diff + B_hist_analysis_diff
R_n_hist_analysis_diff = R_hist_analysis_diff / hist_analysis_total
G_n_hist_analysis_diff = G_hist_analysis_diff / hist_analysis_total
B_n_hist_analysis_diff = B_hist_analysis_diff / hist_analysis_total
```

The second histogram method looks for unusually tall spikes. Manipulations like brightness and contrast changes can pile many pixels into a few values, which makes the mode higher than usual. For each channel, I divided the pixel count at the mode of a channel's histogram by the average pixel count for the 19 highest peaks in the histogram below the mode. A large score means the mode stands out a lot from the rest of the histogram, which suggests tampering. As in the previous method, I normalized the ratios by dividing each by the total, so they fall between 1 and zero and sum to 1.

Code:

```
% Saves all of the histogram values for each channel and the bins they belong to in the
%histogram.
[countsR, binsR] = imhist(R);
[countsG, binsG] = imhist(G);
[countsB, binsB] = imhist(B);
% Saves the max 20 of each channel and their index in separate values
[maxCountR, bin_indexR] = maxk(countsR,20);
[maxCountG, bin_indexG] = maxk(countsG,20);
[maxCountB, bin_indexB] = maxk(countsB,20);

% Gets the ratio of the number of pixels in the highest value in the
%histogram and the other top 20 values averaged for each channel.
R_spike = maxCountR(1) / mean(maxCountR(2:20));
G_spike = maxCountG(1) / mean(maxCountG(2:20));
B_spike = maxCountB(1) / mean(maxCountB(2:20));
% Normalizes the score so the score of each channel sums to 1.
total_spike_score = R_spike + G_spike + B_spike;
R_spike_score = R_spike / total_spike_score
G_spike_score = G_spike / total_spike_score
B_spike_score = B_spike / total_spike_score
% Arbitrary weights added to hist analysis score
R_hist_analysis_score = R_n_hist_analysis_diff*.5 + R_spike_score*.5
G_hist_analysis_score = G_n_hist_analysis_diff*.5 + G_spike_score*.5
B_hist_analysis_score = B_n_hist_analysis_diff*.5 + B_spike_score*.5
```

Noise Estimation:

The third test compares the amount of noise in each channel. All channels of an unedited photo should have similar noise levels. To estimate a channel's amount of noise, I blurred all channels with a Gaussian filter to remove some noise from the channel and took the difference between the original channel and the filtered channel. This should leave us with an image that retains noise and fine details. Taking the standard deviation of this would give us the noise score, where a higher standard deviation means more noise. The three noise scores are then normalized by dividing each by their total, so they sum to 1.

Code:

```
gaussian_kernel = fspecial('gaussian', [7 7],1.5);
R_denoise = imfilter(R,gaussian_kernel,"replicate");
G_denoise = imfilter(G,gaussian_kernel,"replicate");
B_denoise = imfilter(B,gaussian_kernel,"replicate");
% Calculates noise score by finding the standard deviation's of the distance between filtered
%channel and the original channel
R_noise_score = std2(R - R_denoise);
G_noise_score = std2(G - G_denoise);
```

```

B_noise_score = std2(B - B_denoise);
% Normalize noise score
total_noise_score = R_noise_score + G_noise_score + B_noise_score;
R_n_noise_score = R_noise_score / total_noise_score
G_n_noise_score = G_noise_score / total_noise_score
B_n_noise_score = B_noise_score / total_noise_score
% Each test contributes 1/3 to the total confidence score
R_Confidence = R_hist_analysis_score * (2/3) + R_n_noise_score * (1/3)
G_Confidence = G_hist_analysis_score * (2/3) + G_n_noise_score * (1/3)
B_Confidence = B_hist_analysis_score * (2/3) + B_n_noise_score * (1/3)
% Compares Confidence values to determine which channel is most likely
%to be tampered with and return it with the correct confidence score.
if(R_Confidence > G_Confidence && R_Confidence > B_Confidence)
    manipulated_channel = 'R';
    confidence = R_Confidence;
elseif(G_Confidence > R_Confidence && G_Confidence > B_Confidence)
    manipulated_channel = 'G';
    confidence = G_Confidence;
else
    manipulated_channel = 'B';
    confidence = B_Confidence;
end

```

Problem 1b) Restoring the manipulated channel

To correct the manipulated channel, I started with a noise-reducing strategy, but noticed that it was ineffective for shifted or heavily altered channels. To address this, I incorporated a weighted average using the two reference channels. The combination of inter-channel difference smoothing and the weighted average worked well for non-inverted manipulated channels. Lastly, I added a separate check and correction step specifically for inverted channels, detecting them using dynamic highlight/shadow masks. I got the idea from the in-class saliency lab.

Inter-Channel Difference Filtering and Reconstruction:

To deal with added noise, this simple restoration algorithm uses the natural correlation between color channels. For each pixel, I computed difference-maps between the manipulated channel and each reference channel: $d_1 = \text{manipulated} - \text{reference}_1$ and $d_2 = \text{manipulated} - \text{reference}_2$. Since inter-channel color differences should vary smoothly across an image, I applied a Gaussian filter ($\sigma = 5$) to smooth out potential noise and artifacts in both difference-maps: $d_{1,\text{smooth}} = \text{imfilter}(d_1, \dots)$ and $d_{2,\text{smooth}} = \text{imfilter}(d_2, \dots)$. Initial reconstructions are formed by adding these smoothed difference-maps back to their respective reference channels: $\text{sol}_1 = d_{1,\text{smooth}} + \text{reference}_1$ and $\text{sol}_2 = d_{2,\text{smooth}} + \text{reference}_2$. To align the reconstructed channel's overall distribution with the uncorrupted channels (largely to account for shifts), these results are blended in with the two reference channels to create the final output: $\text{restored_channel} = (\frac{1}{3})\text{sol}_1 + (\frac{1}{3})\text{sol}_2 + (\frac{1}{3})\text{reference}_1 + (\frac{1}{3})\text{reference}_2$.

Code (restored_channel.m):

```
function restored_channel = restore_channel(manipulated, ref1, ref2)
    manipulated = double(manipulated);
    ref1 = double(ref1);
    ref2 = double(ref2);

    if is_inverted(manipulated, ref1, ref2)
        restored_channel = 1-manipulated;
    else
        % Differences between bad channel and good channels
        dif1 = manipulated-ref1; % manipulated = dif1 + ref1
        dif2 = manipulated-ref2;

        % try gaussian filter to smooth out differences
        sigma = 5; k = 2*ceil(3*sigma)+1;
        gaussian_kernel = fspecial('gaussian', [k k], sigma);
        dif1_smooth = imfilter(dif1, gaussian_kernel, 'replicate');
        dif2_smooth = imfilter(dif2, gaussian_kernel, 'replicate');

        % solutions
        sol1 = dif1_smooth + ref1;
        sol2 = dif2_smooth + ref2;
        sol = sol1/2 + sol2/2;

        % average this solution with the two good channels
        % to produce a distribution closer to those channels
        restored_channel = sol/3 + ref1/3 + ref2/3;
    end
end
```

Channel Inversion Detection:

If a channel has been inverted, the previous methods are very ineffective. To identify an inverted channel, dynamic thresholds are calculated using the mean (μ) and standard deviation (σ) across the reference channels to isolate highlight and shadow masks (the brightest and darkest regions of the image). The reference masks are calculated as $shadows = (reference_1 < \mu - \sigma) \wedge (reference_2 < \mu - \sigma)$ and $highlights = (reference_1 > \mu + \sigma) \wedge (reference_2 > \mu + \sigma)$. Corresponding shadow and highlight masks are constructed for the manipulated channel using its own mean (μ_{bad}) and standard deviation (σ_{bad}). The function then compares the direct overlap of matching regions against an inverted overlap where shadows match highlights. If the inverted match-count (*inverted_points*) exceeds the normal match-count (*good_points*), the channel is deemed to be inverted and the function returns true.

Code (is_inverted.m):

```
function result = is_inverted(manipulated,ref1,ref2)
    % mean and standard dev of good channels and bad one
    m = mean(ref1(:))/2 + mean(ref2(:))/2;
    s = std(ref1(:))/2 + std(ref2(:))/2;
    m_bad = mean(manipulated(:));
    s_bad = std(manipulated(:));

    % creating masks for highlights and shadows of good channels
    % using thresholds dependent on mean and std
    low = m-s*1;
    high = m+s*1;
    low_bad = m_bad-s_bad*1;
    high_bad = m_bad+s_bad*1;
    shadows = ref1<low & ref2<low;
    highlights = ref1>high & ref2>high;
    shadows_bad = manipulated<low_bad;
    highlights_bad = manipulated>high_bad;

    % see how much the shadows and highlights match between good and bad
    % channels, compare to inverse
    good_points = sum(sum(shadows & shadows_bad)) + sum(sum(highlights & highlights_bad));
    inverted_points = sum(sum(shadows & highlights_bad)) + sum(sum(highlights & shadows_bad));

    % temp
    %figure,imshow(shadows);
    %figure,imshow(highlights);

    result = good_points<inverted_points;
End
```

The result is returned, and an inverted channel is restored simply by flipping the pixel values back to normal, as seen below.

Code (snippet from restored_channel.m):

```
if is_inverted(manipulated,ref1,ref2)
    restored_channel = 1-manipulated;
else
    ...
end
```

Problem 2:

Problem 2) Noise reduction in depth maps

The second portion of the project requests that we denoise a number of depth maps by using bilateral filtering and the maps' associated images. The solution I ended at was to use the more complete images to determine the kernel and kernel sizes that needed to be applied to the depth maps.

So, as such, this portion of the bilateral filter takes the original image, and uses that to determine the range kernels

```
for i = indent + 1:height-indent
    for j = indent + 1:width-indent
        range_kernel = createRangeKernel(img,i,j,kernel_size,sigma_range);
        kernel = (range_kernel.*gaussian_kernel);
```

Followed up by applying those kernels to the maps

```
normalization = 1/sum(kernel(:));
    temp = (kernel.*double(depthImg(i-indent:i+indent,j-indent:j+indent)))*normalization;
    img_results(i,j) = sum(temp(:));
end
end
```

Albeit, the original image has to be ran through the bilateral filter on all three channels, otherwise it was prone to returning the original image in three different tones of grayscale

The following code fixes that:

```
%The filter's adjusted to apply the kernels from the primary image
%and apply them to the depth map
%along with adjusting for multiple color channels
img_results(:, :, c) = bilateralFilter(imgCell{1,i}(:, :, c), depthImgCell{1,i});
```

Results

Problem 1:

Tampering Detection Results:

	IMG 1	IMG 2	IMG 3	IMG 4	IMG 5	IMG 6	IMG 7	IMG 8	IMG 9	IMG 10
R Score	40.11%	23.15%	31.89%	25.42%	23.58%	31.90%	54.82%	23.00%	55.87%	39.82%
G Score	30.54%	51.15%	40.14%	51.50%	54.89%	31.25%	23.18%	53.19%	22.33%	29.39%
B Score	29.35%	25.70%	27.97%	23.08%	21.52%	36.84%	22.00%	23.80%	21.80%	30.79%
Tampered Channel	R	G	G	G	G	B	R	G	R	R
Confidence score	40.11%	51.15%	40.14%	51.50%	54.89%	36.84%	54.82%	53.19%	55.87%	39.82%

Analysis:

The detector identified the correct channel in all 10 images (100% accuracy). However, the confidence scores are quite modest. Because each channel's score is normalized so that R, G, and B sum to 100%, an uninformative result would give each channel about 33%. Scores between 36% and 40% (images 1,3,6 and 10) are only slightly above that, while the 51-56% scores show much more of a separation.

The weakest cases were images 1, 6, and 10, where the image had an inverted channel. This is because inverting the image only inverts the histogram, but the spike test and noise test are largely unaffected by this manipulation. The only test we implemented that is somewhat reliable is the shape test in those cases, but even this could fail in the event of a symmetrical histogram. This means that although this method is sufficient for the test cases, another test is needed to check those outliers.

The spike test was most effective on brightness and contrast changes because they tended to cause large spikes at 0 or 255. But in cases like image 3, the histogram was shifted right, but the histogram did not clip that much at 255, which caused the spike test to have not much influence on the score compared to other images with the same manipulation. This left only the shape test being reliable in this case.

The noise test also contributed little to the final score in all test cases, as none of the images were very noisy and all three channels had similar noise levels.

Another possible way to improve the scores is to weight each test by how informative it is. For example, a test whose scores are all 33% should have little to no weight in the final confidence scores.

Restoring Images Results:



Manipulated green channel of image 4, corrected channel



Manipulated (inverted) blue channel of image 6, reference highlight, reference shadow, manipulated highlight, manipulated shadow, corrected channel

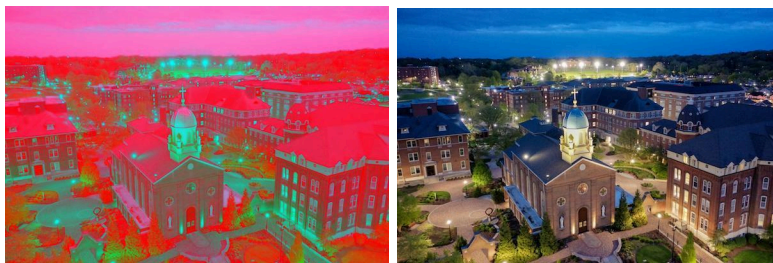


Image 1 constructed with manipulated (inverted) red channel, construction with corrected channel

Analysis:

Overall, the restoration method was very effective at fixing both non-inverted (e.g., shifted) and inverted channels across the test cases.

For standard, non-inverted channels (like the green channel in image 4), only using basic noise reduction was ineffective when pixel values were heavily shifted. Using inter-channel difference filtering smoothed out the noise while preserving details. Blending those smoothed difference solutions (sol_1 and sol_2) with a weighted average of the two reference channels ($(\frac{1}{3})ref_1 + (\frac{1}{3})ref_2$) brought the manipulated channel's overall distribution closer to the rest of the image.

For inverted channels (like in images 1 and 6), difference smoothing failed because the inverted pixels were too far from their original values. The highlight and shadow mask check worked well here. By setting thresholds based on the mean and standard deviation ($\mu \pm \sigma$), the algorithm flagged inverted channels whenever dark areas in the reference channels matched bright areas in the manipulated channel. Once detected, simply flipping the values back ($1 - \text{manipulated}$) completely restored the original channel.

Problem 2:

Bilateral Filtering Results:



Analysis:

Overall, while the depth map is certainly denoised, it also gains large white splotches that follow the edges of the RGB photos. Standard denoising methods would result in a less grainy, but still very dark image. The resulting image does also still follow the few lines that can be seen in the depth image, but bilaterally filtering it makes it moderately clearer as to what it was supposed to be a picture of by highlighting minor splotches that a person wouldn't normally be able to see.

Team Contribution Details

Benjamin Merrell:

- Completed Problem 1a and helped with 1b.
- Wrote the report details and slides for problem 1a.
- Wrote the Readme.txt file.

Elijah Rose:

- Completed Problem 1b and helped with 1a.
- Wrote the report details and slides for problem 1b.
- Compiled resulting images from problem 1 into the results folder.

Ashley Blankenship

- Completed Problem 2
- Wrote the report details and slides for problem 2
- Compiled the resulting images from problem 2 into the test results folder

Team Member	Code:	Report:	Slides	Total
Benjamin Merrell	33%	33%	33%	33%
Elijah Rose	33%	33%	33%	33%
Ashley Blankenship	33%	33%	33%	33%